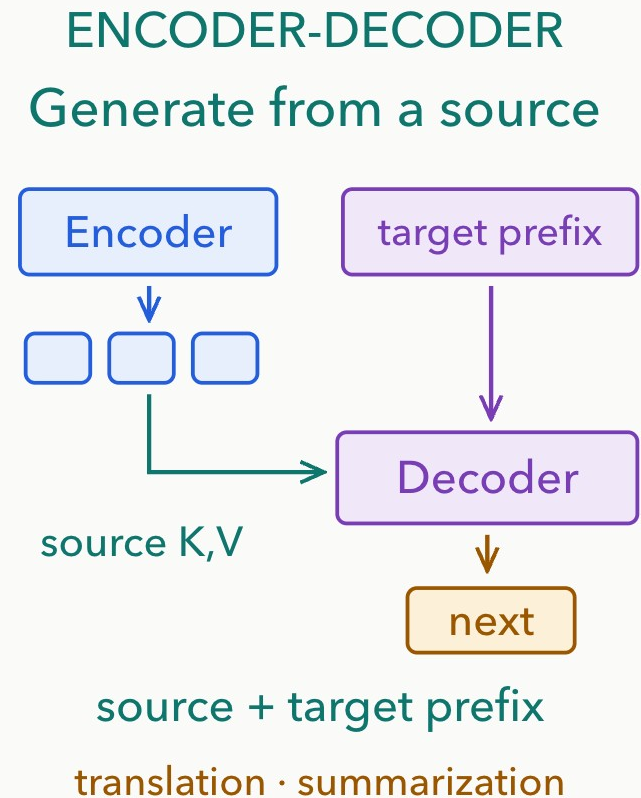
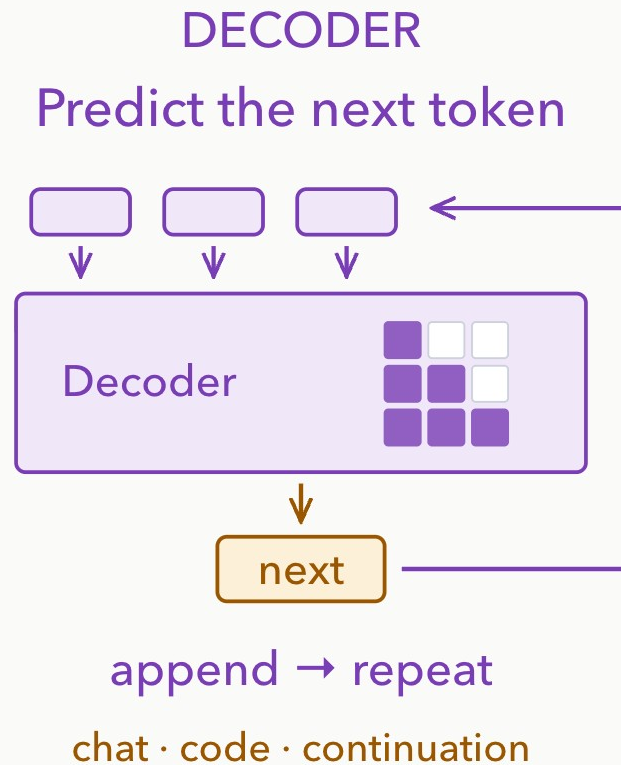
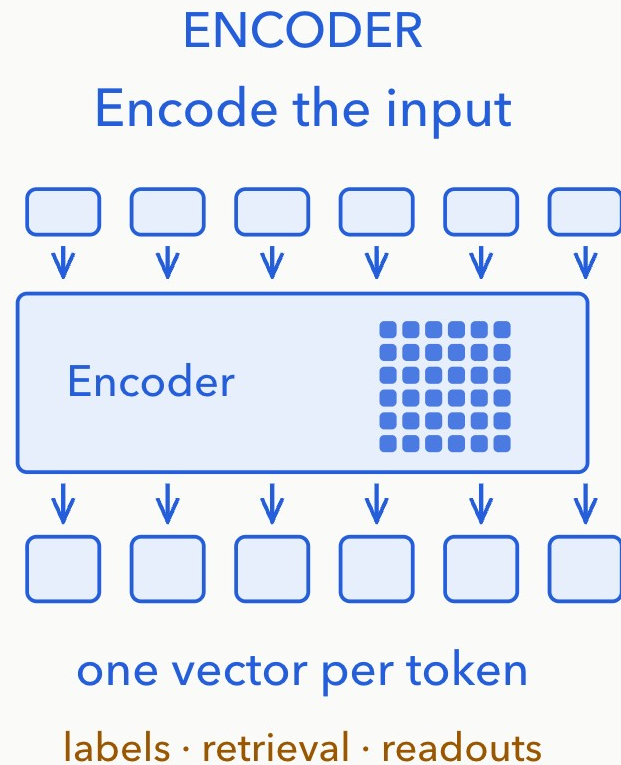


VISION TO LANGUAGE · VISION I

Vision Transformer

Five questions turn one photograph into tokens, context and an ImageNet prediction.

We already know how an encoder builds context



We can reuse the encoder once we turn the image into tokens.

What should the model predict for this photograph?



Our input photograph



One measured forward pass

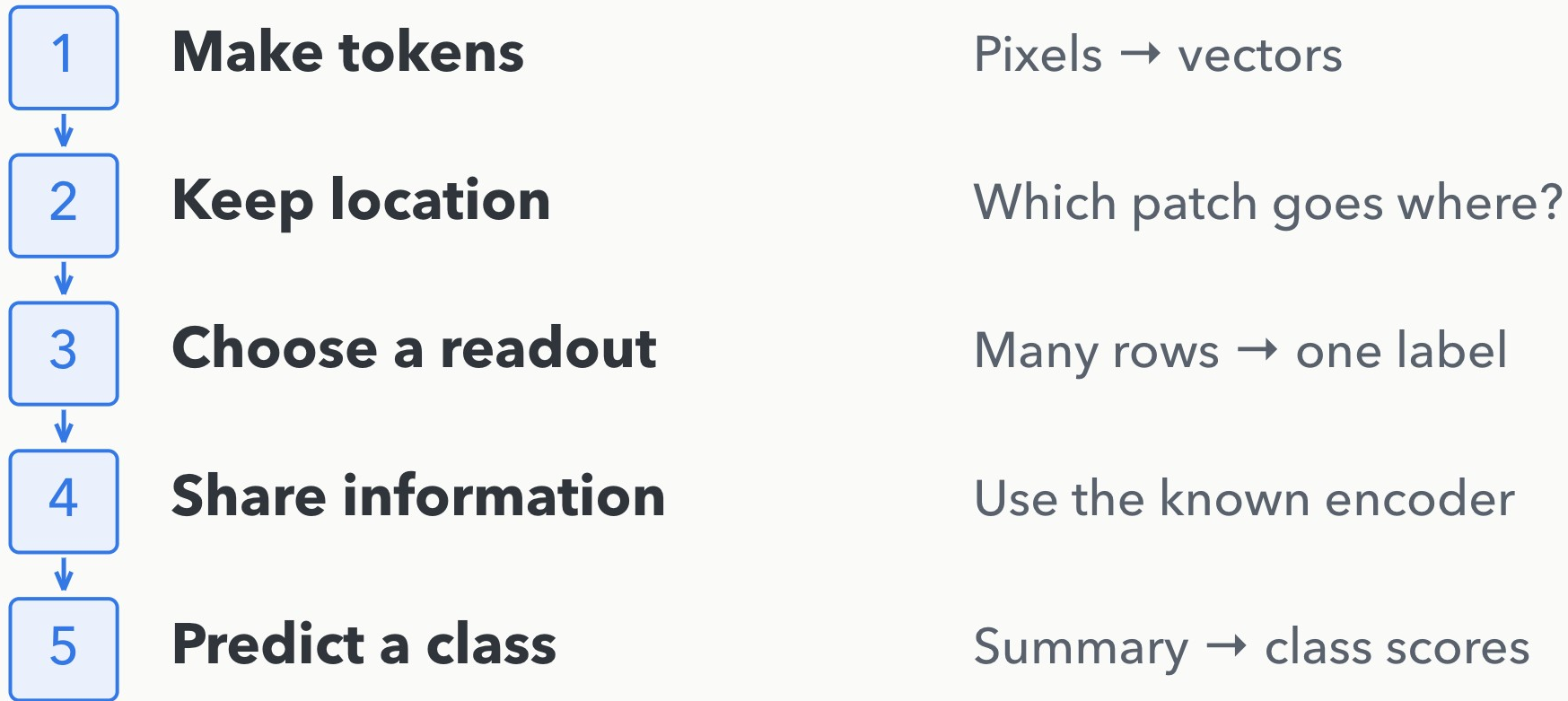
This photograph comes from Oxford-IIIT Pet. The model chooses from 1,000 ImageNet classes.

An encoder expects vectors. How can an image supply them?



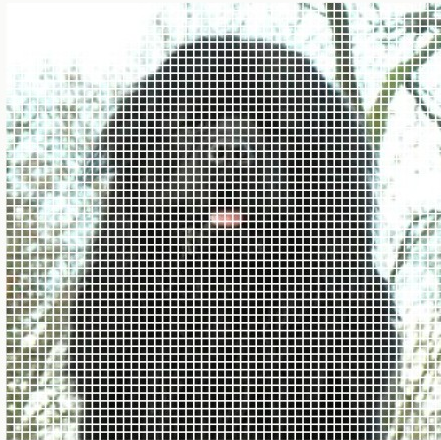
The encoder takes a sequence of vectors. We need to turn the pixels into that sequence.

Five questions build the architecture



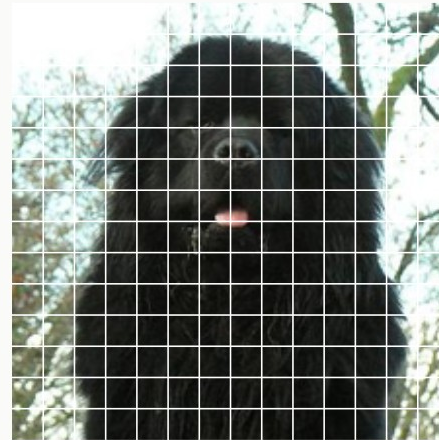
Each question gives us a reason to add the next part of the model.

Why not make every pixel a token?



Pixel tokens

224×224
50,176 tokens



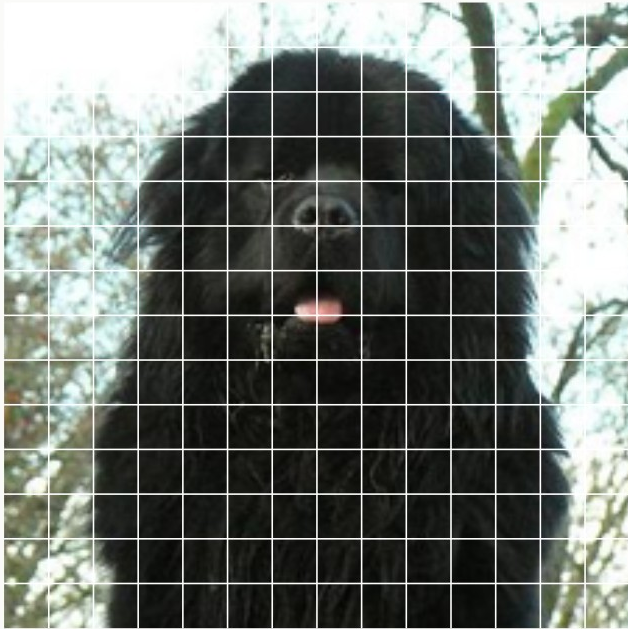
16×16 patches

14×14
196 tokens

Attention compares token pairs: its score matrix grows as N^2 .

Using patches also keeps the number of attention comparisons manageable.

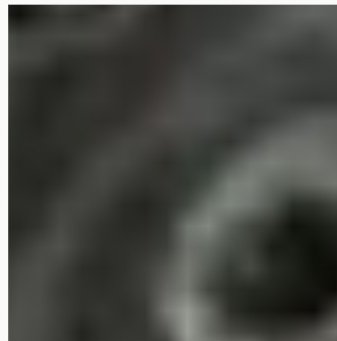
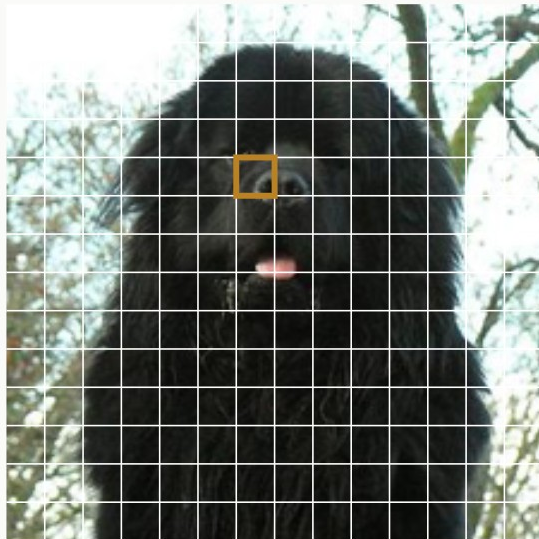
Split the same photograph into 196 patches



$$\begin{aligned} 224 \div 16 &= 14 \text{ patches per side} \\ 14 \times 14 &= 196 \text{ patches} \end{aligned}$$

Each non-overlapping crop is 16×16 pixels, with all three colour channels.

What is inside one patch?



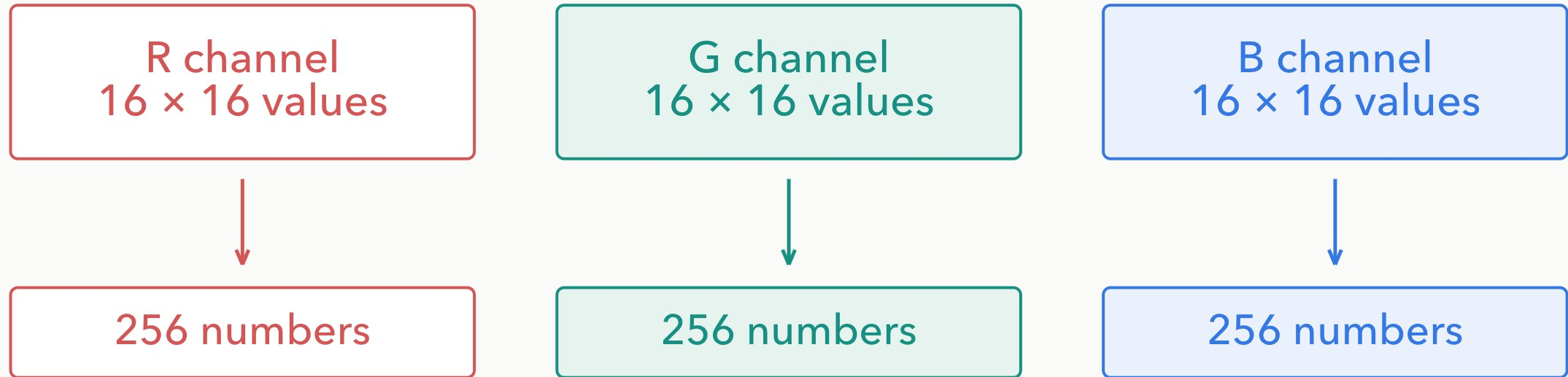
P63



$16 \times 16 \times 3$
768 input numbers

A patch has 768 pixel values. We will add its location separately.

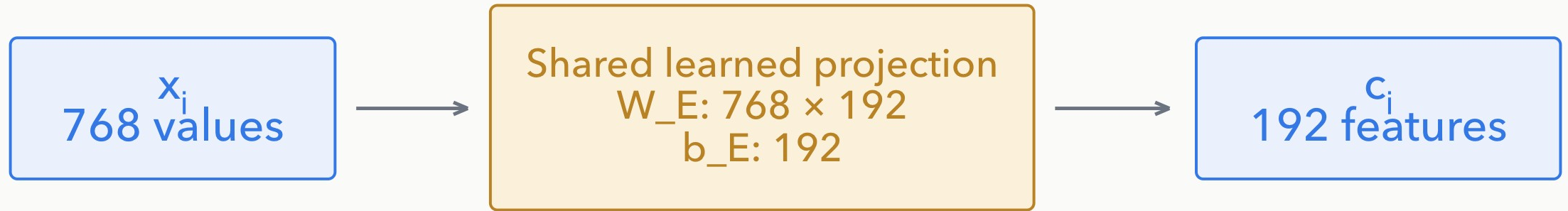
Flatten one channel at a time: R, then G, then B



$$x_i = [\text{all R pixels} \mid \text{all G pixels} \mid \text{all B pixels}] \in \mathbb{R}^{768}$$

Flattening only rearranges the values; it does not learn features.

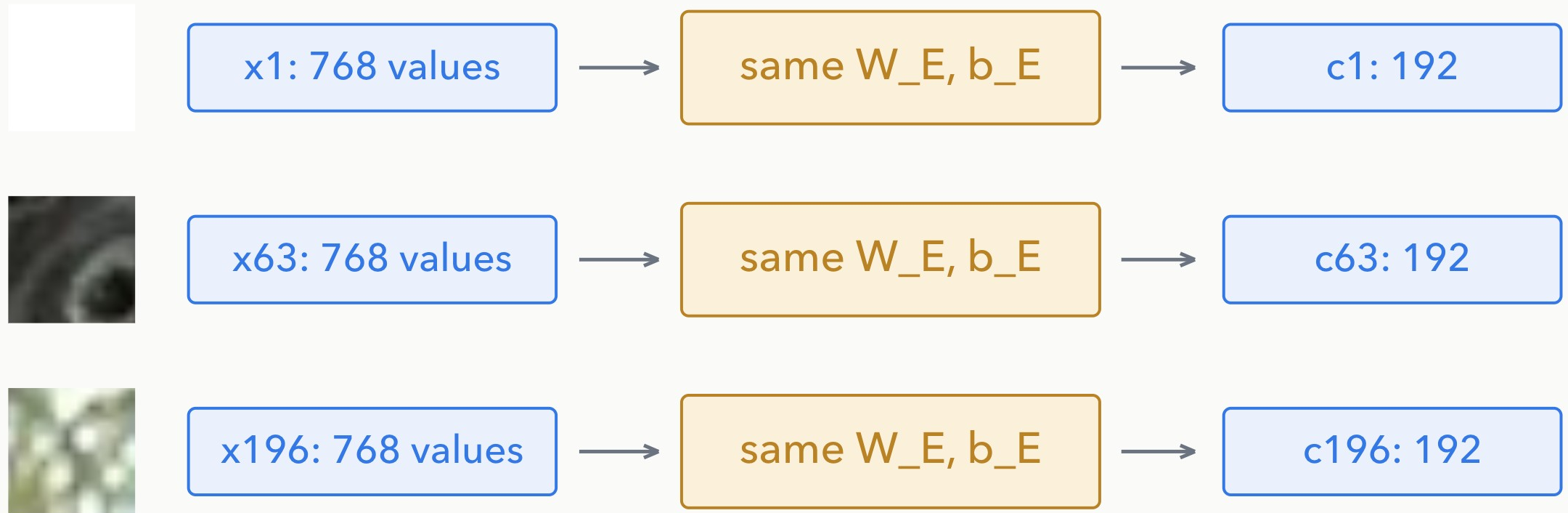
How do 768 pixel values become 192 features?



$$c_i = x_i W_E + b_E$$

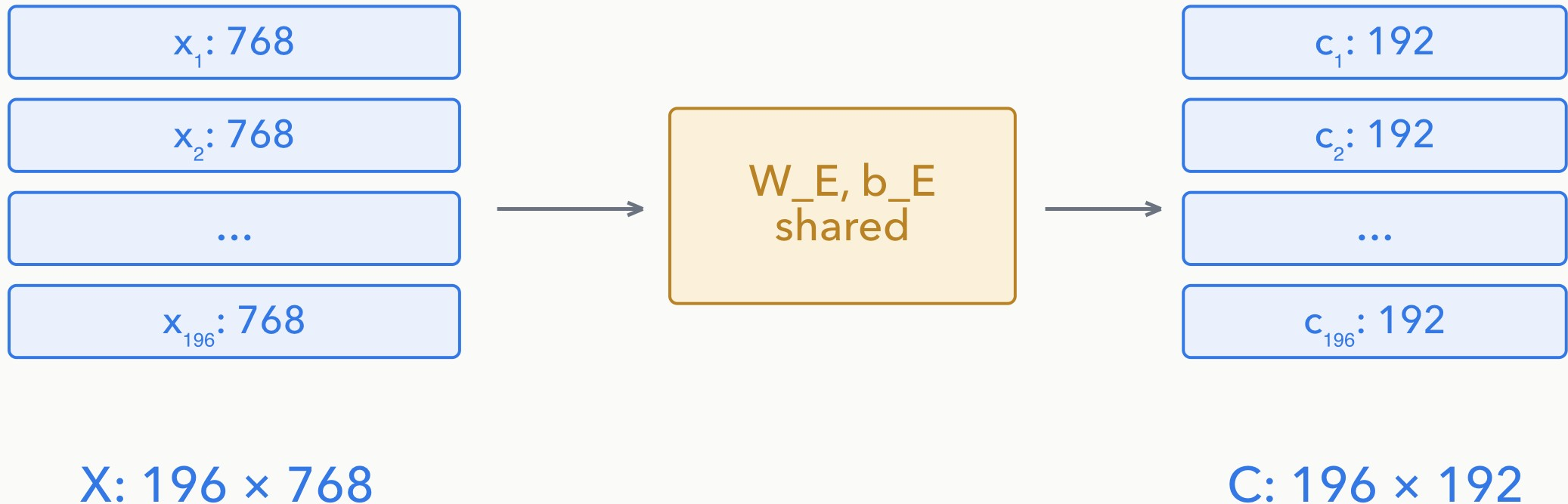
The learned projection turns those 768 pixel values into 192 features.

Should every patch get a different projection?



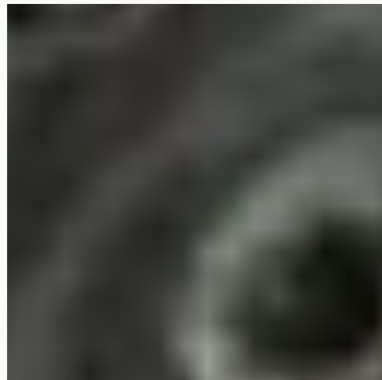
Every patch uses the same W_E and b_E .

Stack the patches into one feature matrix



The projection changes 768 features to 192. We still have 196 patch rows.

What does the trained projection return?



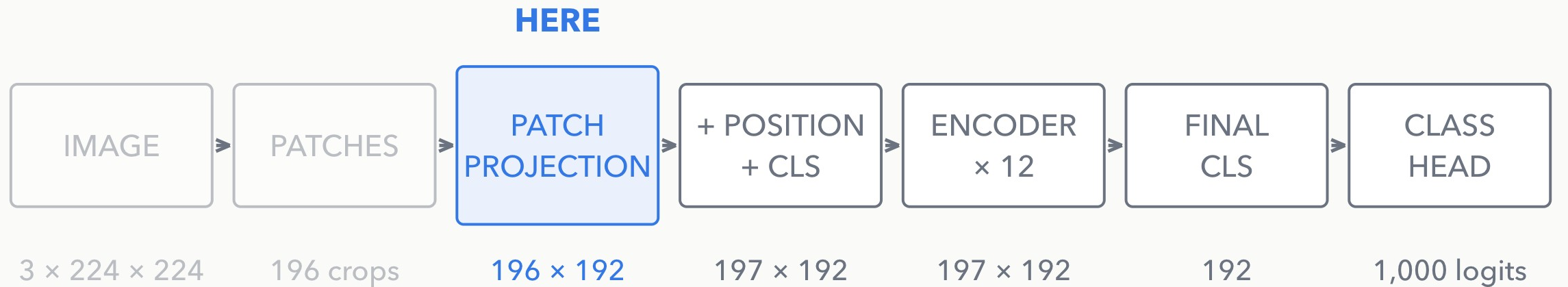
Measured P63



$[-0.852, 1.339, 0.504, \dots]$

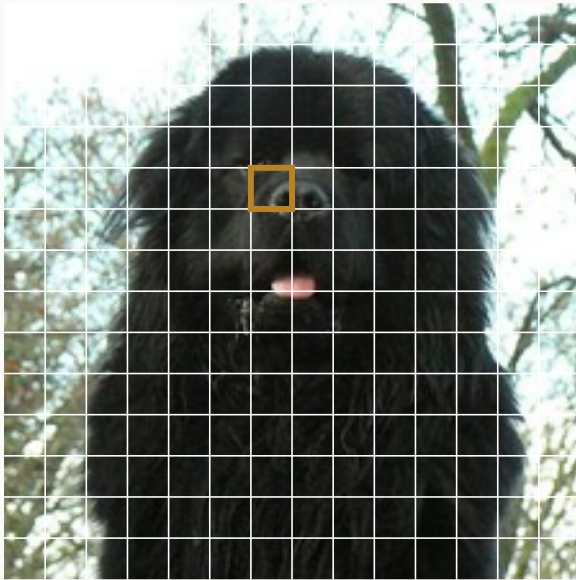
These are the first three of P63's 192 features.

We now have a token for each patch



The image is now 196 rows, each with 192 learned features.

The projection is shared. Where does location enter?



content c_{63}

Where?
row 5, column 7

The content vector describes the crop. A position vector tells the model where it came from.

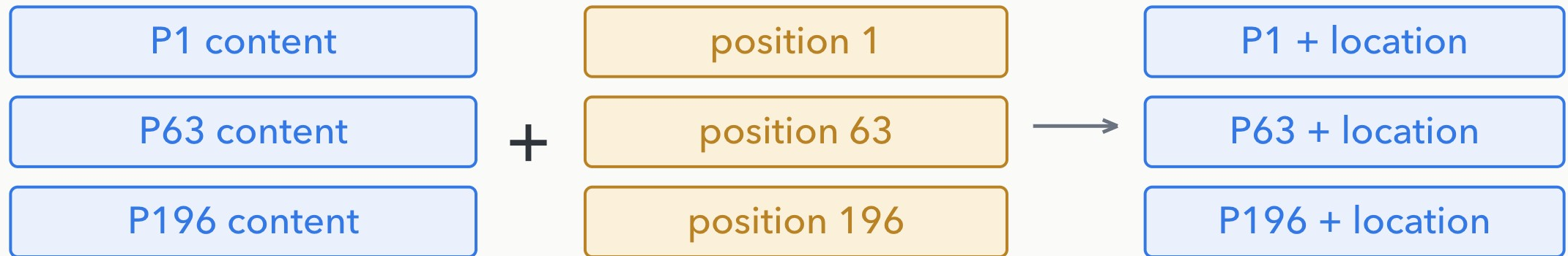
Add WHAT and WHERE



$$e_i = c_i + p_i \cdot 192 \text{ features}$$

Adding position preserves the 196×192 shape.

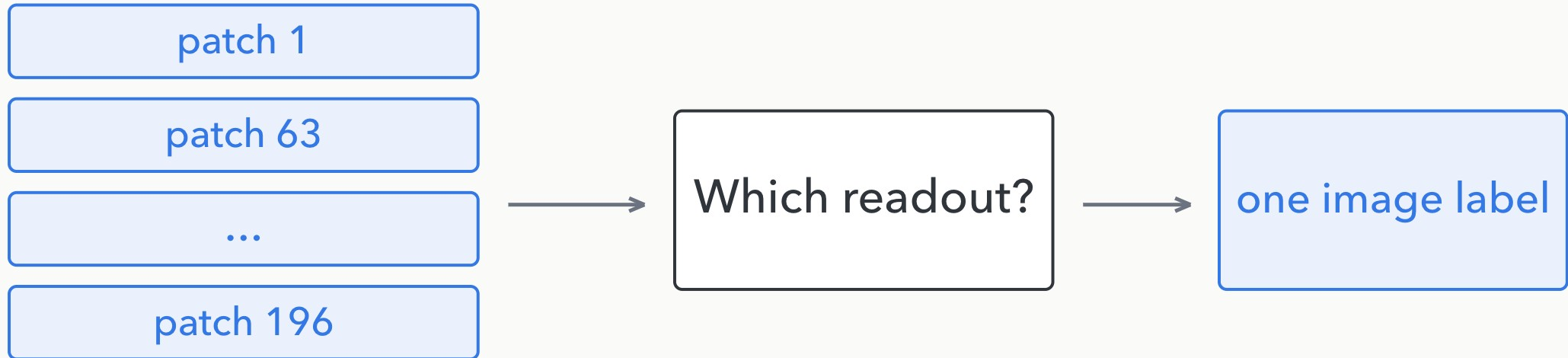
Which positional vector goes with each row?



Learned table: one 192-number row per token position

Every image with this grid uses the same learned position table.

Many patch representations, one image label



Mean pooling is another valid readout; this checkpoint was trained with CLS.

Reuse the CLS readout we already know from text.

Same CLS idea, different input tokens



TEXT

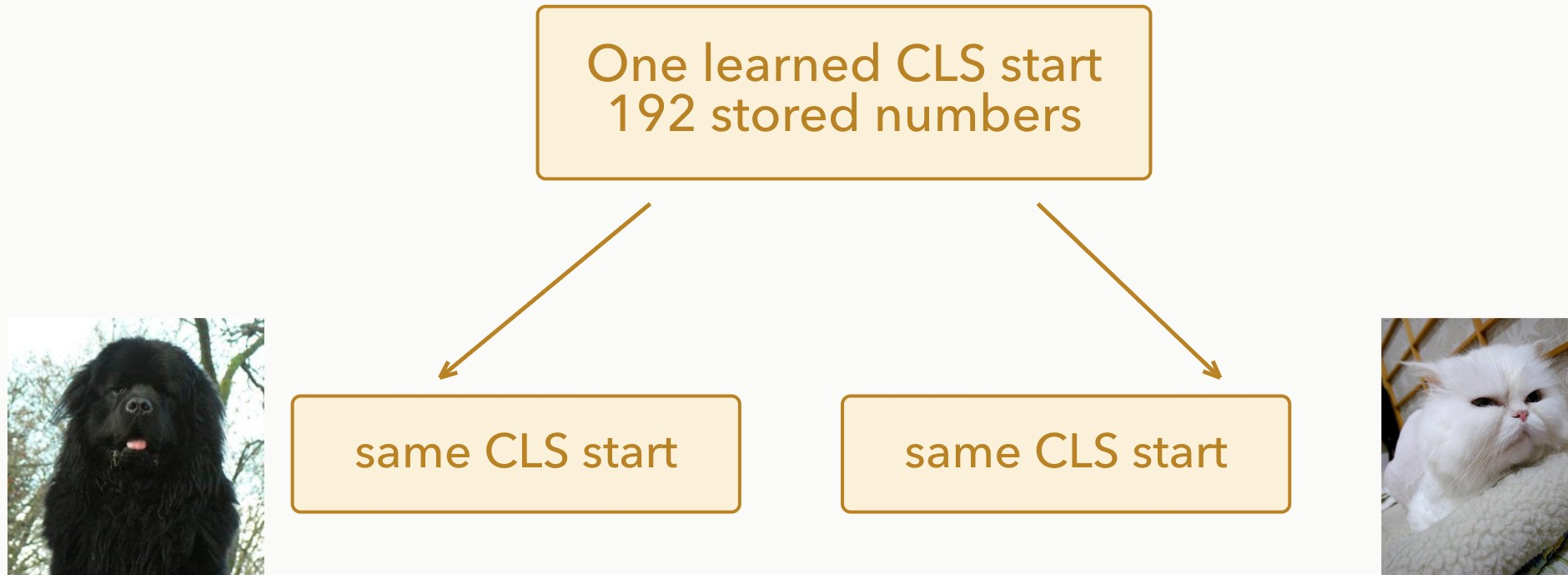


IMAGE



We use patch tokens where the text model used word tokens.

Every image starts with the same learned CLS vector



Both photographs start with exactly the same CLS vector.

After the encoder, CLS depends on the image



same CLS start

same encoder
+ these patch rows

dog image summary



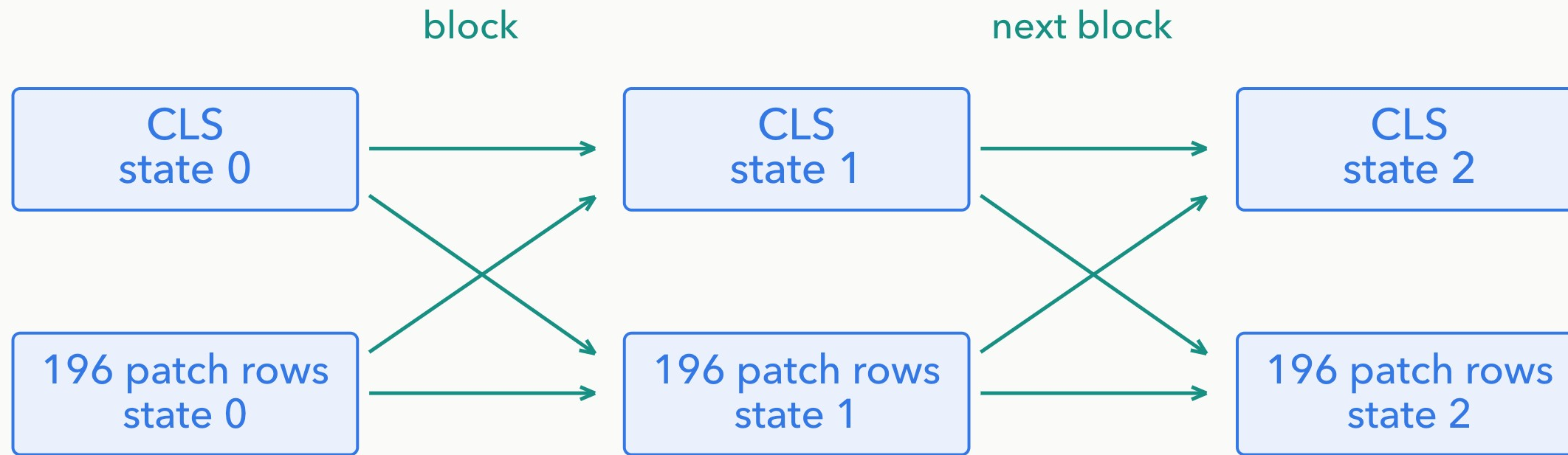
same CLS start

same encoder
+ these patch rows

cat image summary

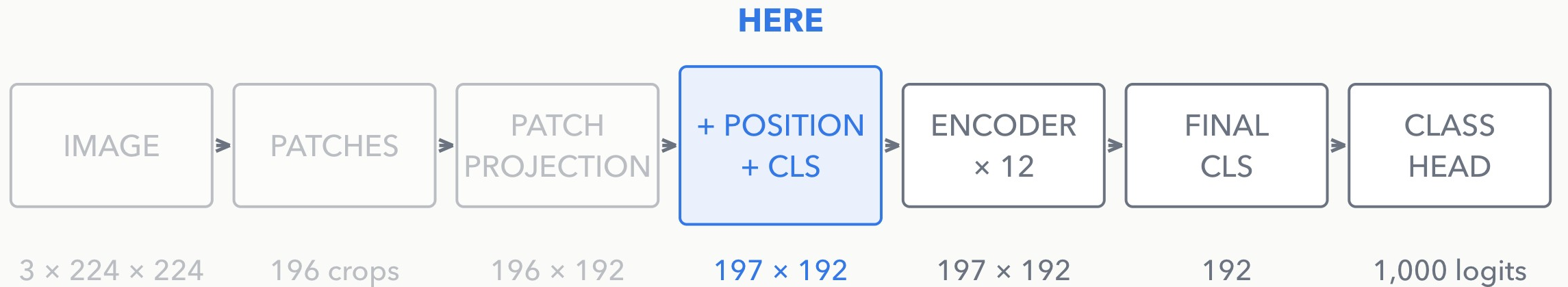
The same model produces a different CLS representation for each photograph.

CLS reads the current patch states at every block



CLS and patch rows can exchange information in both directions.

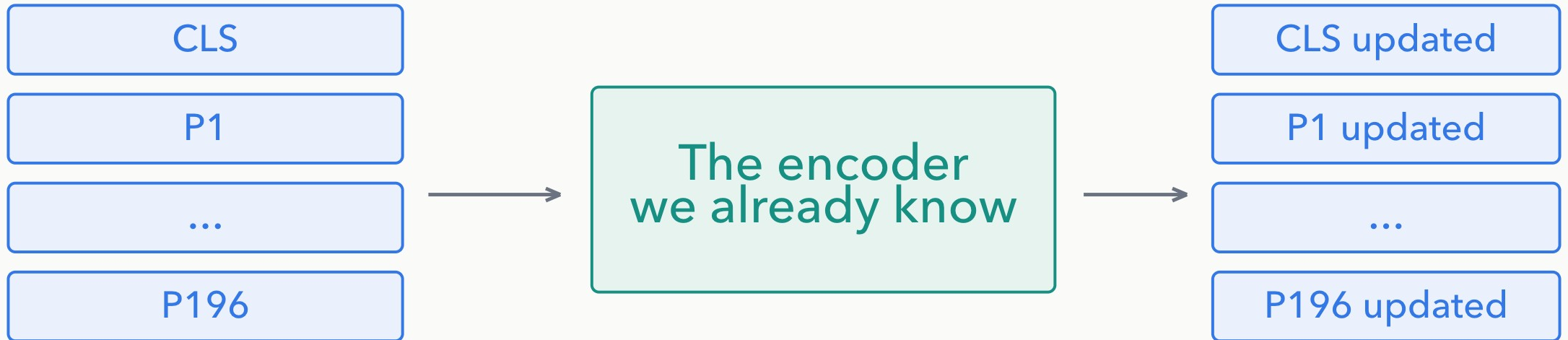
We now have the token sequence the encoder needs



[CLS ; P1 ; P2 ; ... ; P196] + position → 197 × 192

196 projected patches plus one CLS row, each with 192 features.

From here, reuse the encoder we already know



We can now apply the same encoder operations we used for text.

Where do Q, K and V come from?



TRANSLATION • cross-attention

target-prefix states $\rightarrow$ Q

source-encoder states $\rightarrow$ K, V

ViT • self-attention

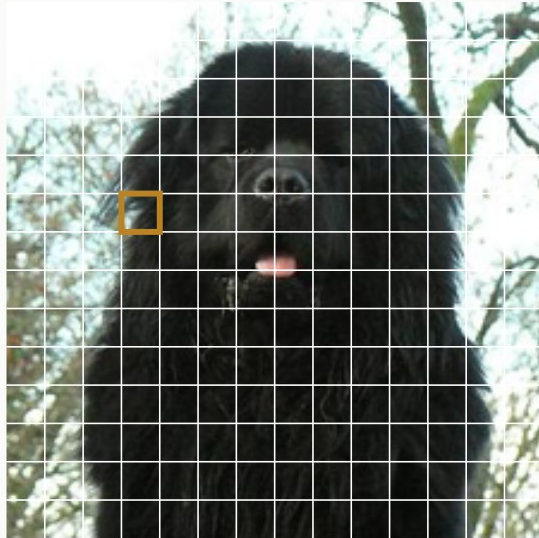
CLS + patch states
one image sequence



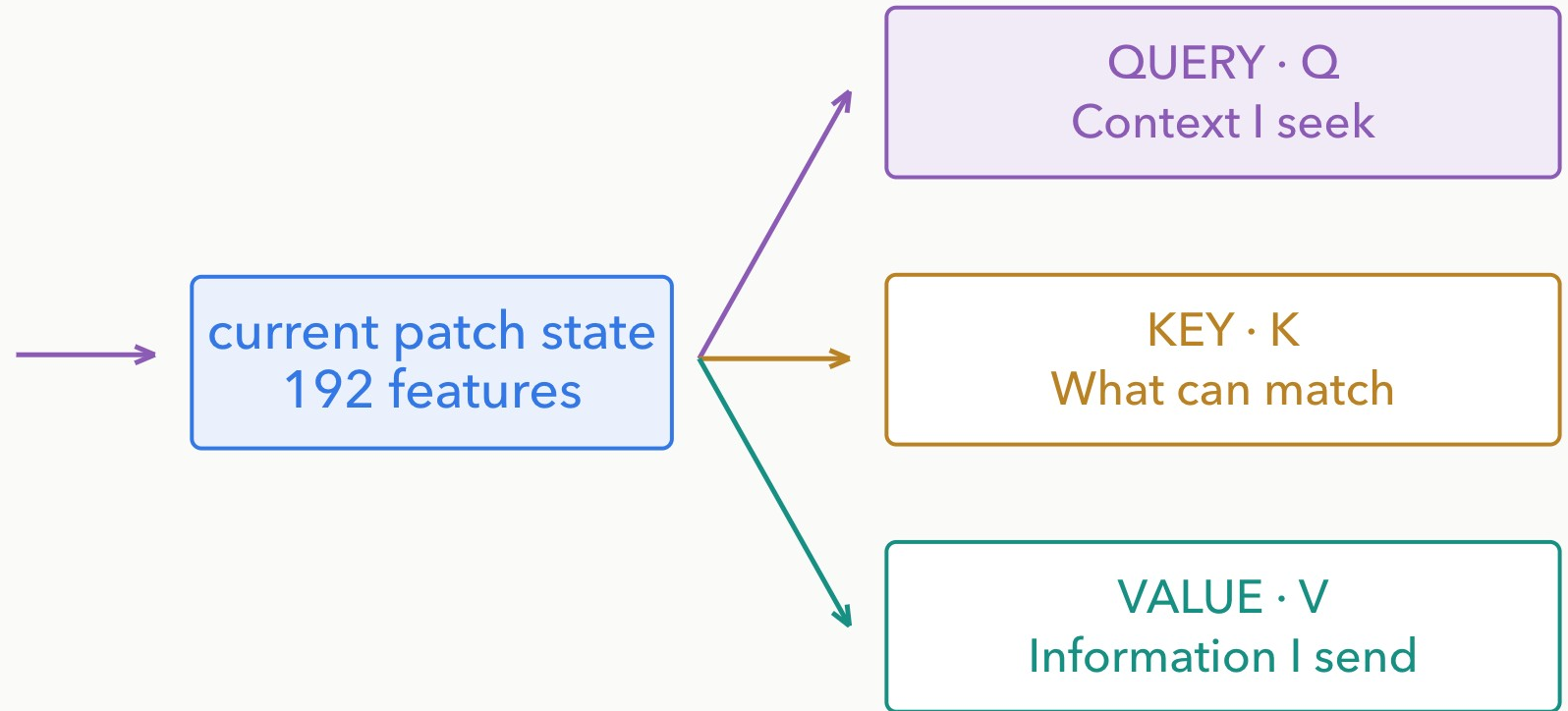
Q, K and V

ViT gets Q, K and V from one image sequence. Translation cross-attention gets them from two sequences.

One patch representation has three jobs



Receiver: P74

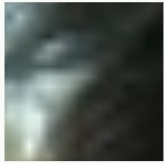


Every token makes a query, a key and a value through three learned projections.

What might different queries try to gather?



Illustrative questions: the model uses vectors, not sentences



A patch on the dog

Which other regions help interpret this texture?

Other fur / face regions



A background patch

Which other regions provide scene context?

Branches / bright background



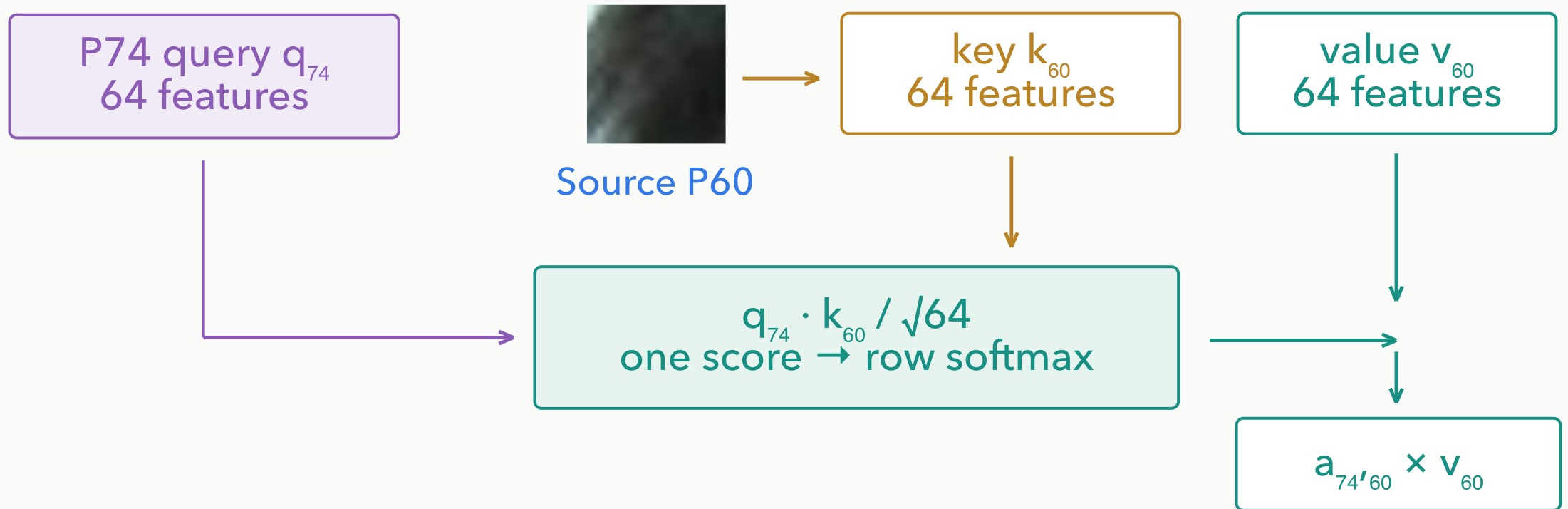
CLS

Which image features help classify the whole image?

Useful parts across the photograph

Each query can gather different information from the same image.

A source supplies both a key and a value



Q and K set a scalar weight. V supplies the vector being weighted.

The weight for P60 scales all 64 coordinates of P60's value vector.

Several source values form one receiver's message



Toy example · one receiver · three sources · two value features



Add the contributions: $m = [1.3, 0.4]$

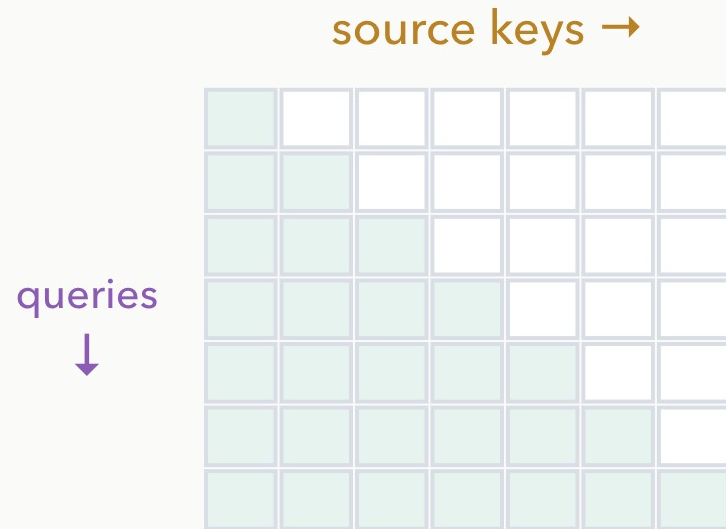
This message goes to the receiver whose query chose the weights.

Multiply each value vector by its weight, then add the results to get one message.

ViT uses full attention, not causal attention



TEXT DECODER • causal



Earlier + current tokens

ViT ENCODER • full

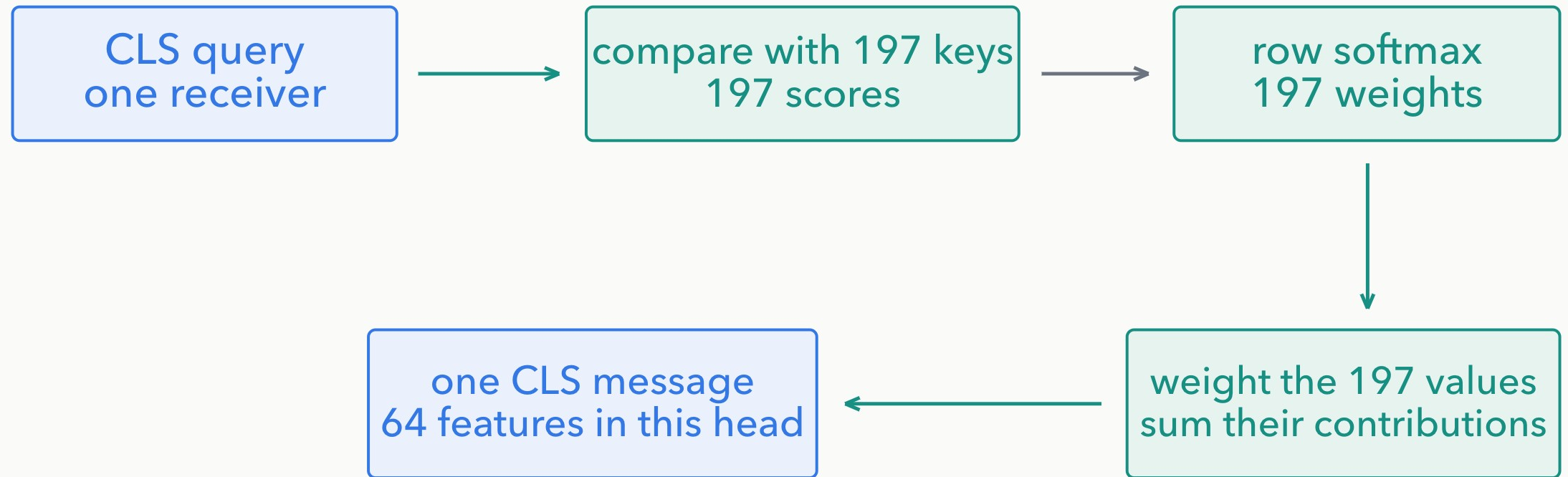


CLS + every patch token

Filled = allowed. ViT sees the complete image: no causal mask.

Each row is a query; each column is a key. Every pair is allowed, but the weights can differ.

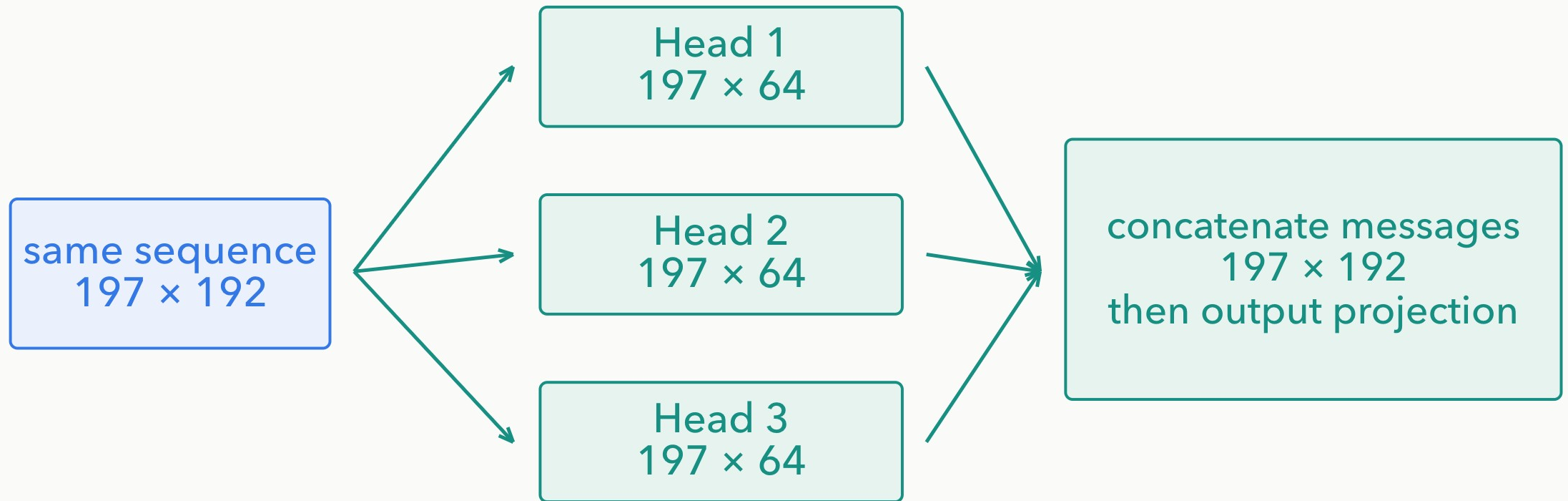
How does one CLS query collect one message?



Other query rows receive their own messages in parallel.

We used the CLS query, so this message updates CLS.

Three heads gather three messages for each token

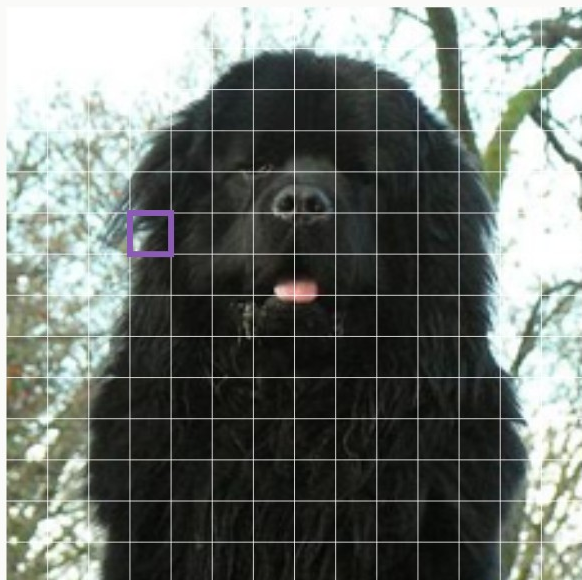


Join each row's three 64-feature messages, then project them back to 192 features.

Where does the trained query look?

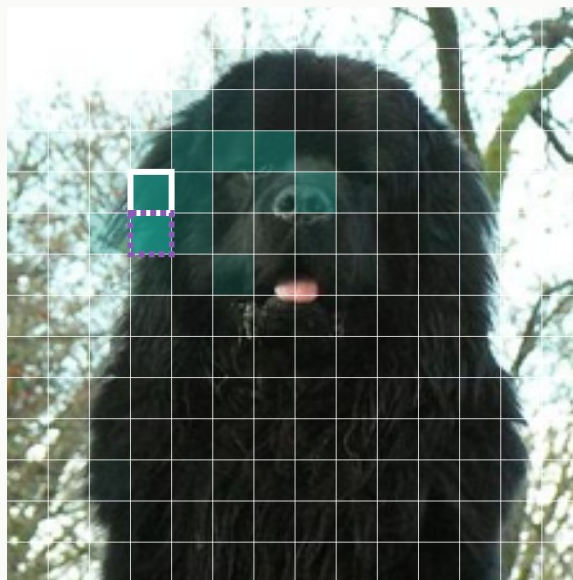


Query: P74

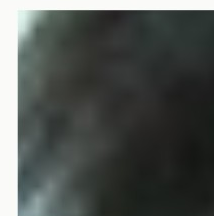


Trained ViT · block 4 · head 1

Measured source weights



Teal: 0 → 10.35%



P60

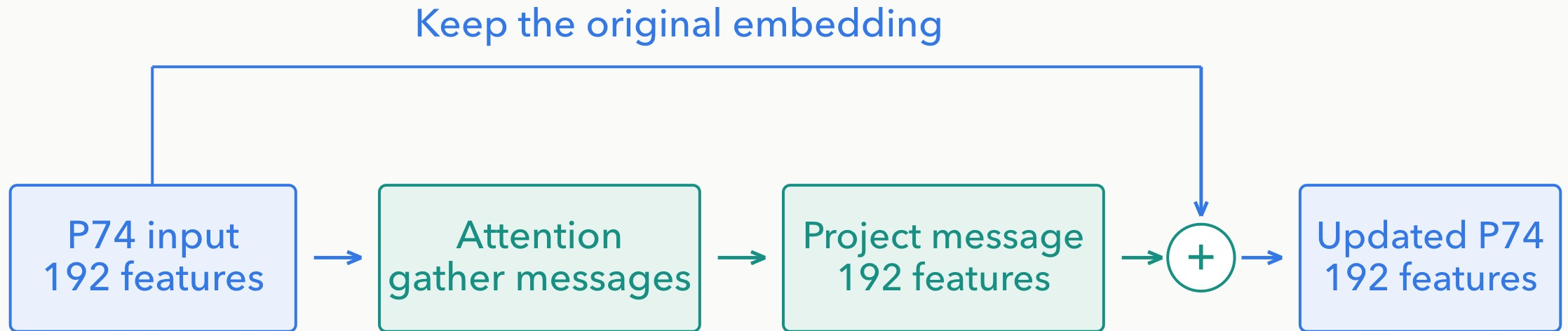
Weight: 9.09%

P60's value contributes
to P74's message.

[Explore the nine examples ↗](#)

Choose one query and look at its source weights. Another head or block may gather a different mixture.

Add the attention update to the original embedding

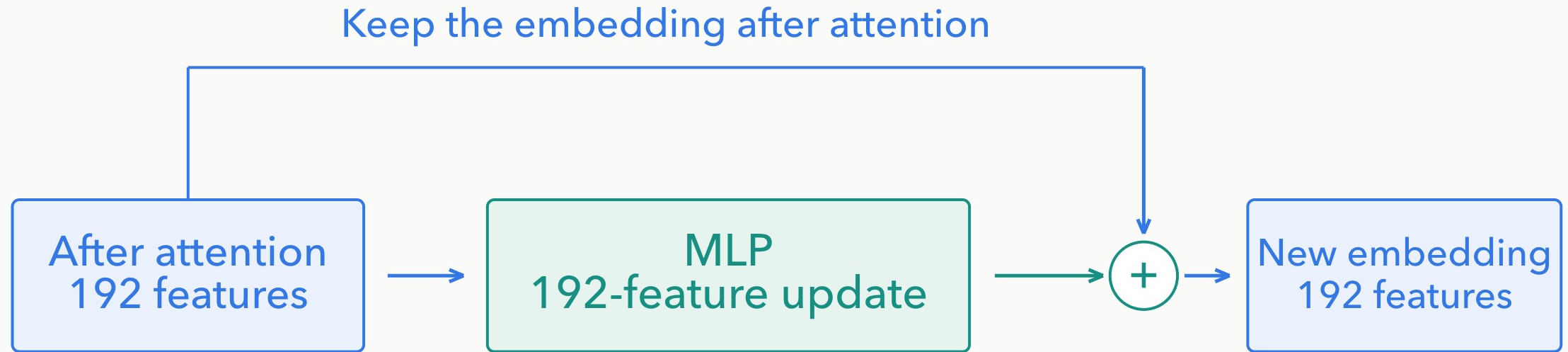


original embedding + projected message = updated embedding

Every patch gets its own update. So does CLS.

Gather a message, project it to 192 features, then add it to the receiving token's original embedding.

The MLP adds one more update to each embedding



The MLP processes each token separately.

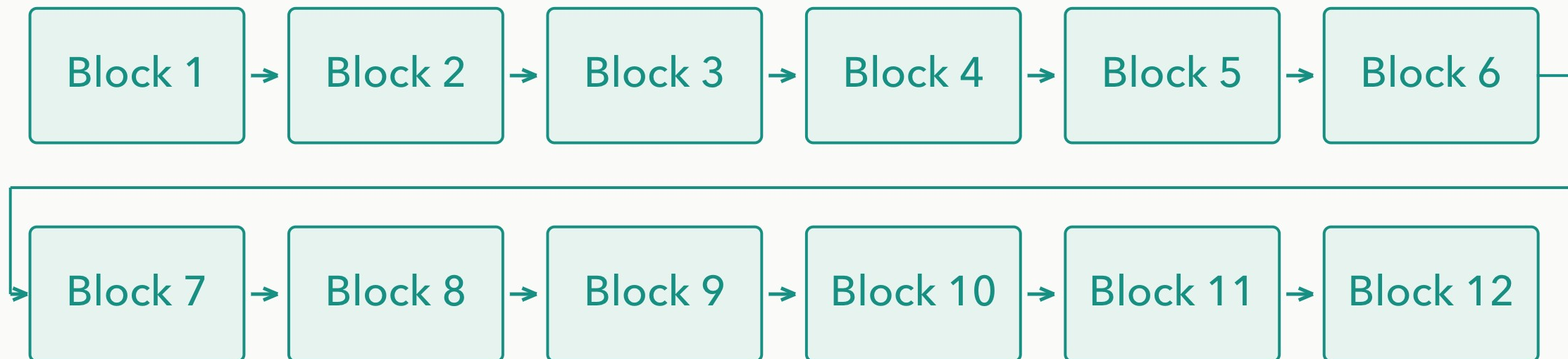
This new embedding is ready for the next block.

Attention gathers context from other rows; the MLP then processes each row's features.

Every block updates the patches and CLS again



Each block: attention update, then MLP update.



197 token embeddings · still 192 features each

The next block starts from these new embeddings. After block 12, we read CLS.

What is stored, and what changes with the image?



LEARNED PARAMETERS

Patch projection W, b

Position table + starting CLS

Transformer weights, including $W_Q/K/V$

Classifier weights + biases

COMPUTED FOR EACH IMAGE

Patch embeddings

Q, K, V and attention weights

Contextual patch states + final CLS

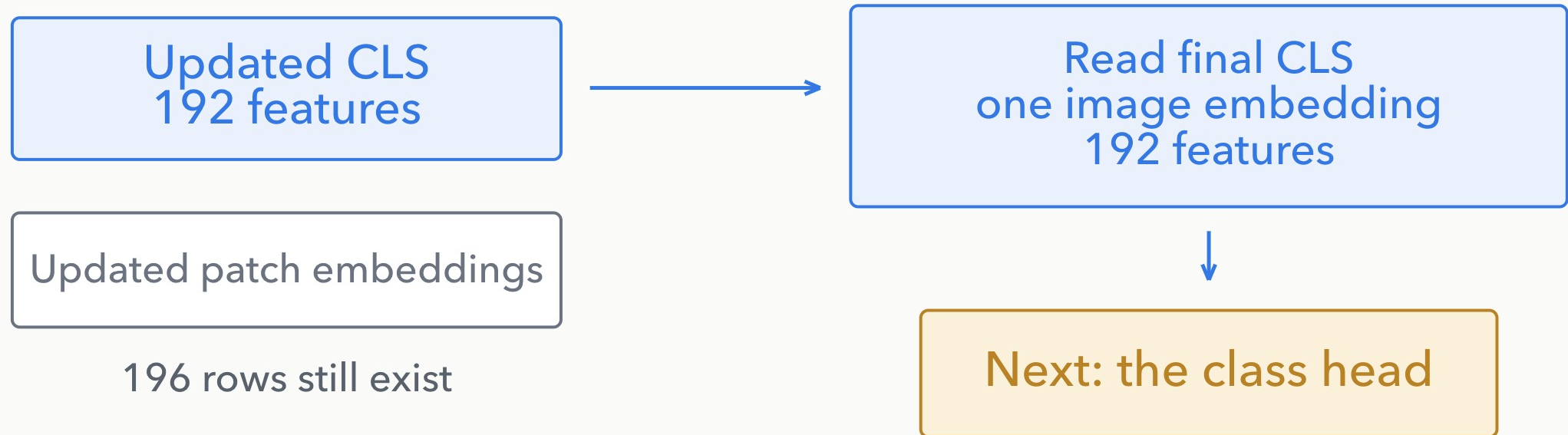
Logits + probabilities

Training learns W_Q, W_K and W_V . Each new image gets its own attention weights.

After the blocks, read the updated CLS embedding



After all 12 blocks



The classifier uses the image summary carried by CLS.

The classifier reads the final CLS embedding as its image summary.

How do 192 features score 1,000 classes?



One logit and one probability for every ImageNet class

The class head learns how each image feature contributes to each class score.

What does the model predict for our photograph?



Newfoundland **95.73%**



Tibetan mastiff **1.63%**



briard **0.67%**

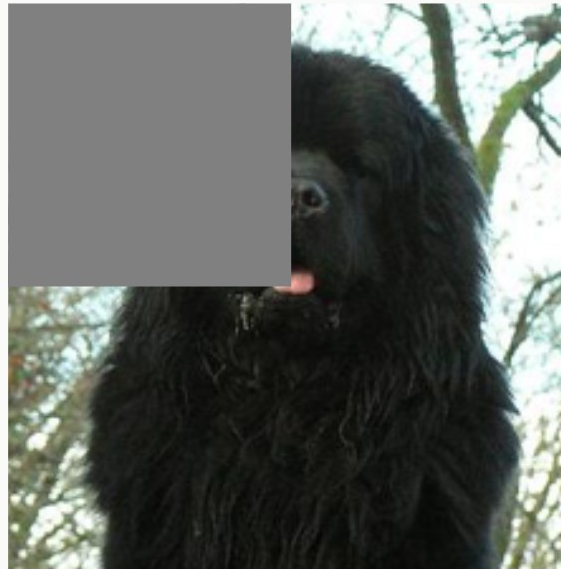
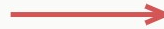


The model assigns Newfoundland a probability of 95.73% among its 1,000 classes.

What happens if we hide one quarter of the image?



Original: 95.73%



Cover one quarter

Same model.
Changed pixels.

Same label?
Same probability?

What do you expect to change when we pass the covered image through the same model?

Change the pixels, then run the model again



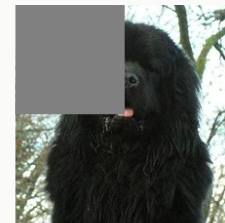
```
covered = x.clone() # x: (1, 3, 224, 224), normalized
covered[:, :, :112, :112] = 0 # gray RGB; 112 x 112 pixels
model.eval()
with torch.inference_mode():
    before = model(x).softmax(-1)[0, 256]
    after = model(covered).softmax(-1)[0, 256]
```



same ViT



95.73%



ViT



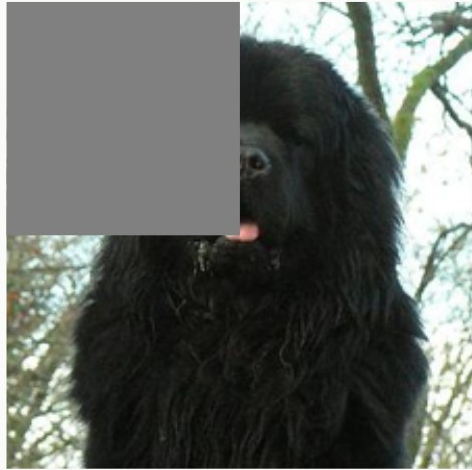
83.00%

The model keeps its learned weights and recomputes patch features, attention weights and class probabilities.

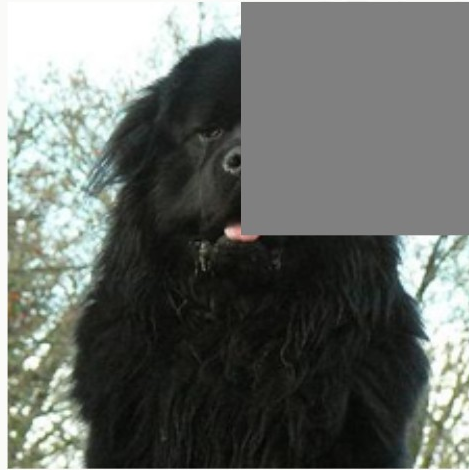
Cover each quarter, then compare predictions



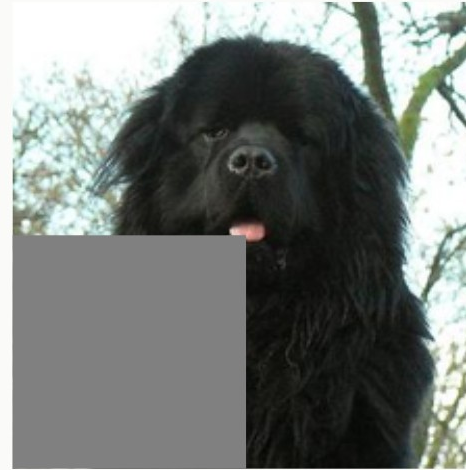
Original $P(\text{Newfoundland}) = 95.73\%$



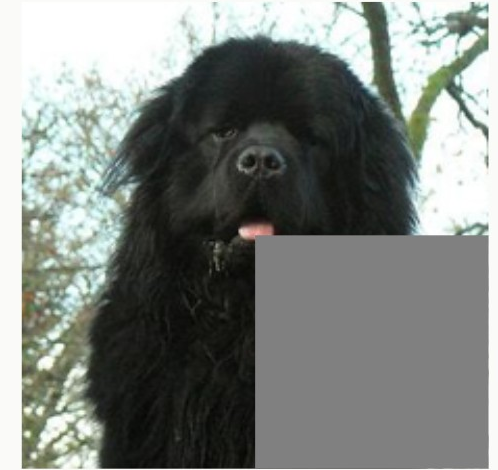
Top left
83.00%
-12.73 points



Top right
79.62%
-16.11 points



Bottom left
84.54%
-11.19 points



Bottom right
82.16%
-13.57 points

Covering the top right gives the biggest drop: 16.11 percentage points. All four covered images still predict Newfoundland.

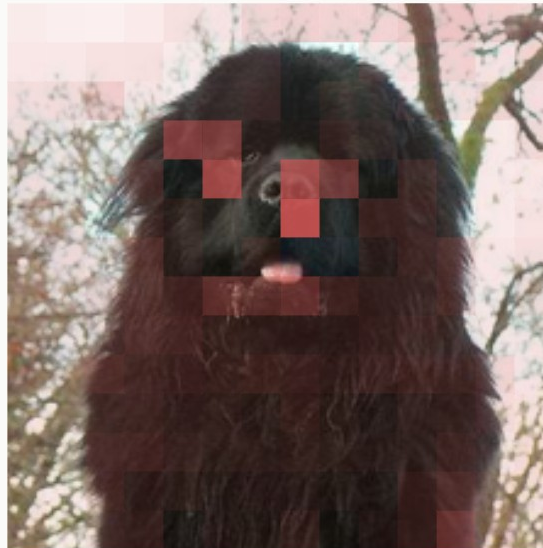
Would smaller covers tell us more?



Cover one patch: P78



196 separate tests



Largest local drop

95.73% → 91.89%

P78: -3.84 points

Red: probability falls

Blue: probability rises

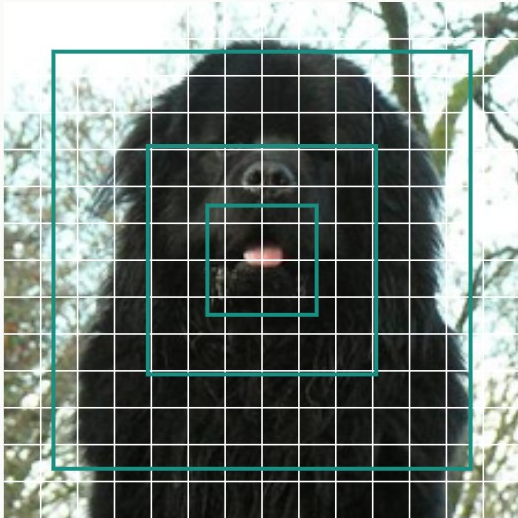
Scale: -4 to +4 points

All 196 tests keep the same label. Smaller covers probe more local sensitivity.

Each coloured square records a separate forward pass with one 16×16 patch covered.

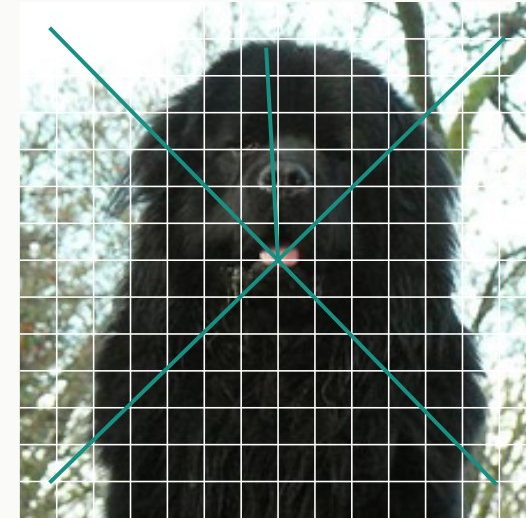
Two ways to gather image context

CNN



Nearby first → wider context

ViT



Direct access to distant patches

CNNs gather wider context through local layers. Global ViT attention can connect distant patches in a single block.

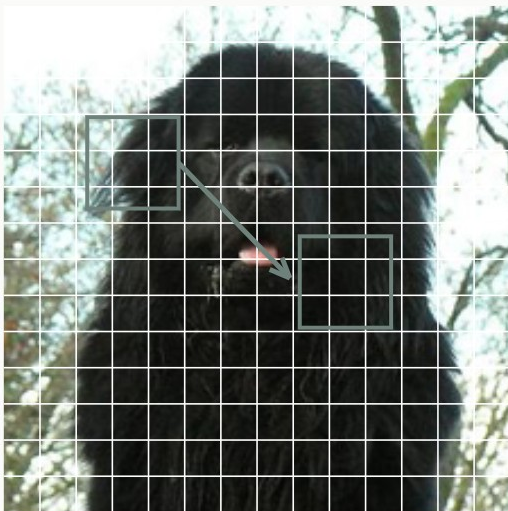
Which assumptions are built into the architecture?

	CNN	ViT
Basic units	Local feature maps	Patch-token rows
Mixing rule	Shared local convolution	Content-dependent attention
Locality	Built into each filter	Less spatial structure built in
Position	Grid structure	Explicit position embeddings

An inductive bias is an assumption built into the model before it learns from data.

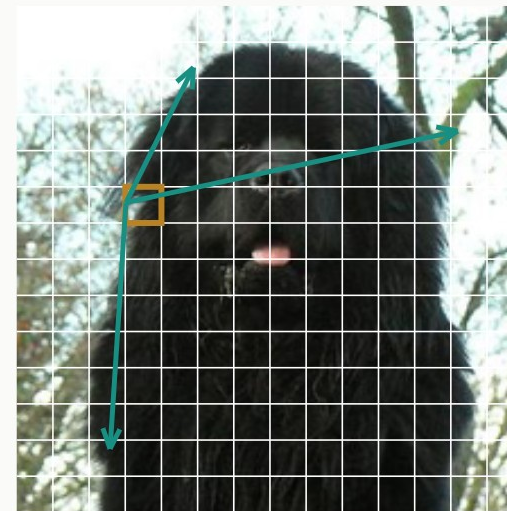
How can these built-in assumptions help?

CNN: reuse a local detector



Same filter, different locations

ViT: choose context from content



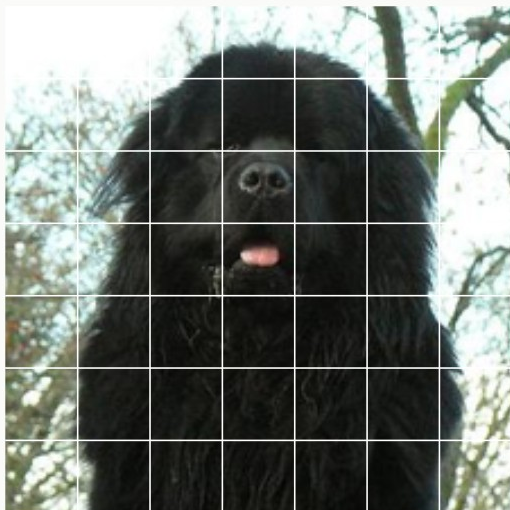
Different weights for each query

A built-in image prior can help with limited data; pretraining changes the comparison. CNN filters assume that nearby pixels matter and that the same pattern can occur in different places.

How much detail should one token cover?

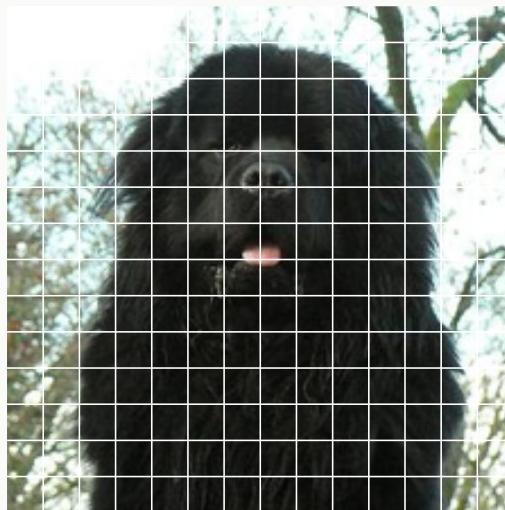


32 × 32 patches



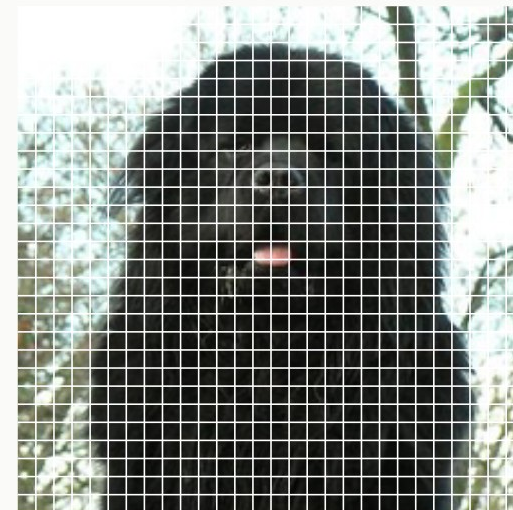
49 patch tokens

16 × 16 patches



196 patch tokens

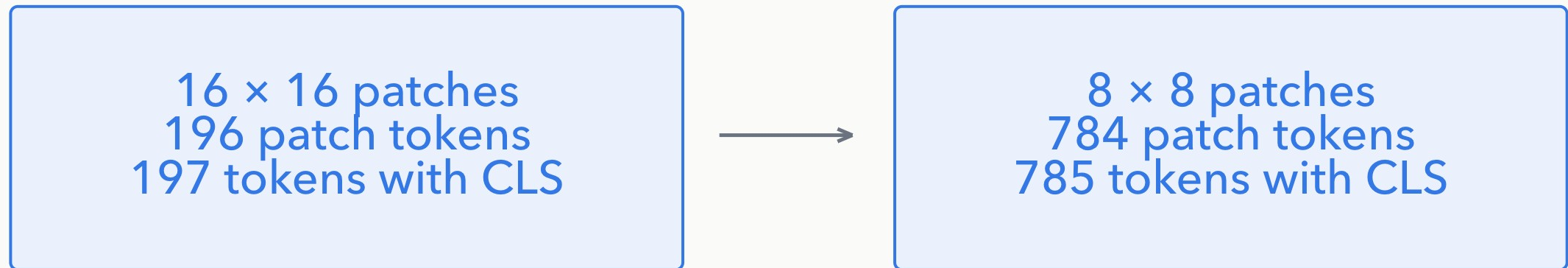
8 × 8 patches



784 patch tokens

Smaller patches give us a finer grid, with more tokens to process.

What does a finer grid cost?

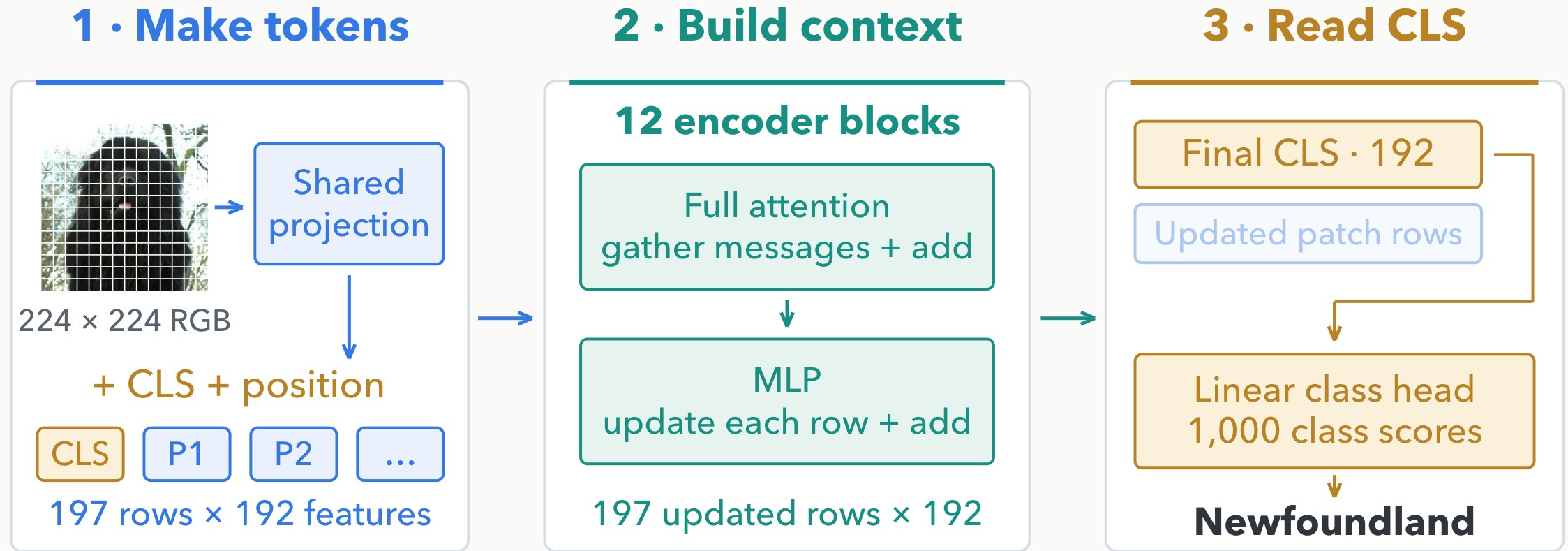


Patch tokens × 4 → attention scores ≈ × 16

38,809 scores → 616,225 scores per head

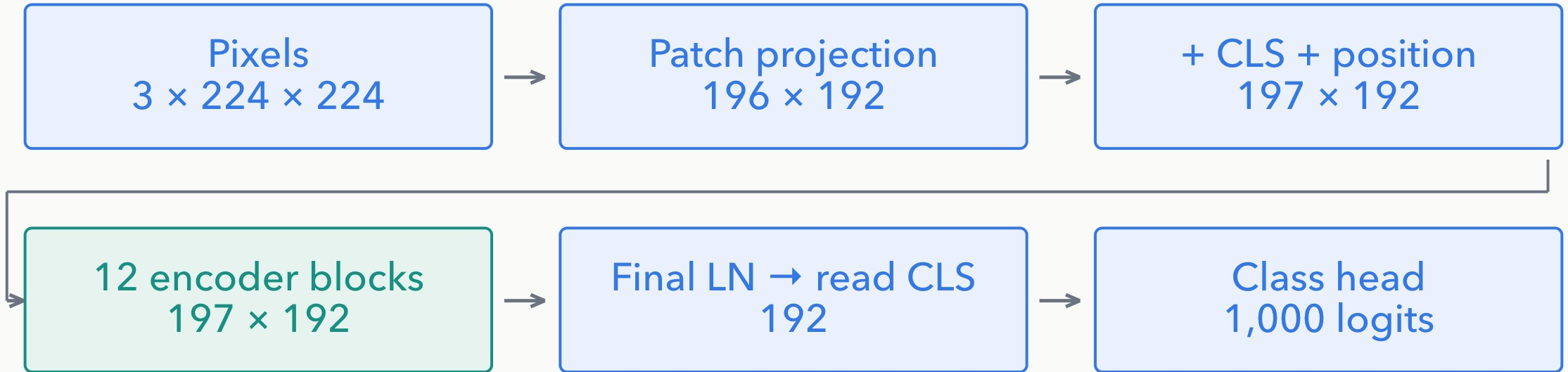
Each token compares with every token, so doubling the token count gives four times as many scores.

Once an image becomes tokens, the encoder is familiar



Turn patches into tokens, update them with the encoder, then use final CLS to predict the class.

Follow the whole model through its shapes



The blocks keep the same shape.
Each row now includes context.

These shapes describe one photograph. The batch dimension is left out.

The whole ViT in six lines

```
# Project each image patch to 192 features.
x = patch_embed(image)           # (B, 196, 192)
# Add CLS at index 0 and position to every token.
x = prepend_cls(x) + pos         # (B, 197, 192)
# Update all patch embeddings and CLS in each block.
for block in blocks:
    x = block(x)                  # (B, 197, 192)
# Normalize, then extract token 0: final CLS.
h = norm(x)[: , 0]               # (B, 192)
# Turn the image summary into 1,000 class scores.
logits = head(h)                  # (B, 1000)
```

B is the batch size. Token 0 is CLS; the head returns scores for the 1,000 ImageNet classes.

More detail, when you want it

OPTIONAL LABS & EXTENSIONS

Implementation lab

Extensions

Open the boxes

Linear $\leftrightarrow$ Conv2d

Transfer + evaluation

Patch arithmetic

PyTorch shapes + code

Similarity · attention · occlusion

Real attention numbers

Use these labs to work through the details.

The reference deck has the full calculations, code, diagrams and exercises.

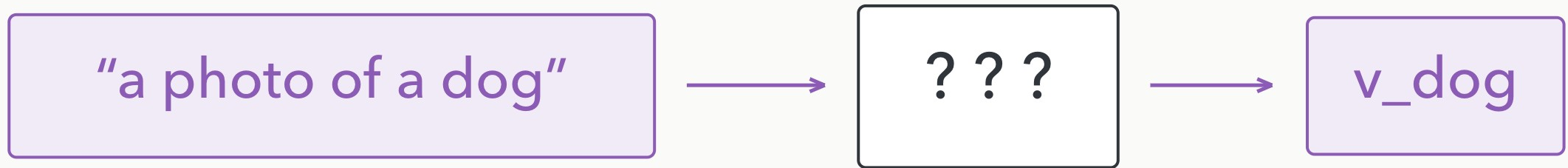
The classifier stores a learned vector per known class



$$s_c = h_{CLS}^T w_c + b_c$$

Its 1,000 output classes form a fixed vocabulary.

What if our class vocabulary could come from words?



What if a class vector came from language?

NEXT: CLIP

We will pick up this question in the CLIP lecture.